

Task 1: Prepare Stock Price Sequence Dataset

Step 1: Import Libraries

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
```

Step 2: Load Dataset

Use a CSV file containing stock closing prices.

Example:

```
df = pd.read_csv("stock_prices.csv")

print(df.head())
```

Example CSV

```
Date,Close
2024-01-01,102
2024-01-02,103
2024-01-03,104
...
```

Step 3: Normalize Data

```
scaler = MinMaxScaler()

prices = scaler.fit_transform(df[['Close']])
```

Step 4: Create Sliding Window

Sequence Length = 10

```
sequence_length = 10
```

```
X = []
```

```
y = []
```

```
for i in range(len(prices)-sequence_length):
```

```
    X.append(prices[i:i+sequence_length])
```

```
y.append(prices[i+sequence_length])
```

```
X = np.array(X)
```

```
y = np.array(y)
```

```
print(X.shape)
```

```
print(y.shape)
```

Example Output

```
(290, 10, 1)
```

```
(290,1)
```

Your dataset is now ready for training.

Task 2: Build and Train GRU Model

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import GRU
```

```
from tensorflow.keras.layers import Dense
```

```
gru_model = Sequential([
```

```
    GRU(
        50,
        input_shape=(10,1)
    ),
```

```
    Dense(1)
```

```
])
```

```
gru_model.compile(
```

```
    optimizer="adam",
```

```
    loss="mse"
```

```
)
```

Train Model

```
import time
```

```
start = time.time()

history_gru = gru_model.fit(

    X,
    y,

    epochs=10,

    batch_size=16,

    verbose=1

)
```

```
gru_time = time.time()-start
```

Since verbose=1, Keras prints the loss after each epoch automatically.

Example

```
Epoch 1/10
loss: 0.0215
```

```
Epoch 2/10
loss: 0.0148
```

```
Epoch 3/10
loss: 0.0097
```

Task 3: Build and Train LSTM Model

```
from tensorflow.keras.layers import LSTM
```

```
lstm_model = Sequential([

    LSTM(
        50,
        input_shape=(10,1)
    ),

    Dense(1)

])
```

```
lstm_model.compile(

    optimizer="adam",

    loss="mse"

)

Train Model

start = time.time()

history_lstm = lstm_model.fit(

    X,
    y,

    epochs=10,

    batch_size=16,

    verbose=1

)

lstm_time = time.time()-start
```

Calculate Final Loss

```
gru_loss = history_gru.history["loss"][-1]

lstm_loss = history_lstm.history["loss"][-1]

print(gru_loss)

print(lstm_loss)
```

For regression tasks such as stock price prediction, **loss (e.g., MSE)** is the standard evaluation metric. If your assignment explicitly asks for "accuracy," you can add a regression metric such as MAE or R^2 , but classification-style accuracy is generally not meaningful for continuous-value prediction.

Task 4: Comparison Table

Parameter Count

```
print("GRU Parameters")
```

```
gru_model.summary()
```

```
print("LSTM Parameters")
```

```
lstm_model.summary()
```

Example Comparison

Model Training Time Final Loss (MSE) Parameters

GRU	2.8 sec	0.0048	7,950
LSTM	3.6 sec	0.0045	10,451

Observation

- GRU trained faster because it has fewer gates and fewer trainable parameters.
- LSTM achieved a slightly lower loss in this example, which may help on more complex or longer sequences.
- For smaller datasets or applications where speed is important, GRU is often a better choice.

Task 5: When Should We Choose GRU?

GRU is generally preferred when:

- Training speed and lower computational cost are important.
- The dataset is relatively small.
- Long-term dependencies are present but not extremely complex.
- Faster inference is required on mobile or embedded devices.

Real-World Applications

Application	Why Choose GRU?
Speech Emotion Recognition	Audio sequences are relatively short, and GRU provides fast training with good performance.

Application**Why Choose GRU?**

Predictive Text /
Keyboard Suggestions

Mobile keyboards require quick inference and low memory usage, making GRU a good fit.

Human Activity
Recognition (Wearables)

Smartwatches and fitness trackers process sensor data efficiently with GRU due to its lightweight architecture.